

Object-Oriented Programming (OOP) in Python

1. How OOP is Different in Python

```
'''  
a. OOP Enforcement: In Python it is optional.  
b. Multiple Inheritance: It is supported '''
```

2. What is a Class?

```
# A class is a blueprint or template.
```

3. What is an Object?

```
# An object is a real instance created from a class.
```

```
class Student:  
    pass  
  
s1 = Student()  
s2 = Student()  
print(s1)  
print(s2)
```

Output:

```
<__main__.Student object at 0x000001941EAEC050>  
<__main__.Student object at 0x000001941E925090>
```

```
#Each object has:
```

```
    #Its own memory  
    #Its own data
```

4. What is a Constructor?

```
'''A constructor is a special method that:  
Runs automatically  
Initializes object data  
  
In Python, it is called:  
__init__()  
  
'''
```

```
#self keyword refers to current instance, similar to this in Java  
class Student:  
    def __init__(self, name, marks):  
        self.name = name  
        self.marks = marks  
  
s1 = Student("Ali", 85)  
s2 = Student("Sara", 92)  
  
print(s1.name, s1.marks)  
print(s2.name, s2.marks)
```

Output:

```
Ali 85  
Sara 92
```

5. Is Constructor Overloading supported in Python?

```
# No if we define multiple init the last will be called  
#Constructor overloading is not directly supported in Python because it does not  
allow multiple __init__() methods in a class.
```

```
class Student:
    def __init__(self,name):
        print("Default constructor")

    def __init__(self,name,age):
        print("Parameterized constructor")

    def __init__(self,name,age,email):
        print("Thid Cpnstructor")

s1= Student("Ali")

#Note: Self keyword is mandatory
```

Error:

`TypeError: Student.__init__() missing 2 required positional arguments: 'age' and 'email'`

#How to acheive?
#*args or **kwargs

```
class Student:
    def __init__(self, *args):
        if len(args) == 0:
            print("No arguments")
        elif len(args) == 1:
            print("One argument")
        elif len(args) == 2:
            print("Two arguments")

s1=Student()
s2=Student(1)
s3=Student(1,2)
```

Output:

No arguments
One argument
Two arguments

6. Instance Data (Instance Variable) vs Class Data (Class Variable)

```
# Instance Data: Data that belongs to a particular object, each object has separate copy.
# Usually inside __init__
#Instance Data → Student Name, Roll No (different for each student)

class Student:
    def __init__(self, name):
        self.name = name      # instance data

s1 = Student("Ali")
s2 = Student("Sara")

print(s1.name)    # Ali
print(s2.name)    # Sara
```

Output:

```
Ali
Sara
```

```
s1.name="Ahmed"
print(s2.name)
#Change in instance data of one object doesn't affect other
```

Output:

```
Sara
```

```
#Class Data (Class Variables): Data that belongs to the class, Shared among all objects.
#Directly inside the class and Outside any method
#Class Data → College Name (same for all students)

class Student:
    school = "ABC School"    # class data

    def __init__(self, name):
        self.name = name

s1 = Student("Ali")
s2 = Student("Sara")

print(s1.school)    # ABC School
print(s2.school)    # ABC School
```

Output:

```
ABC School
ABC School
```

```
#This doesn't modify class variable but instead creates an instance variable for s1
s1.school="XYZ"
print(s1.school)
print(s2.school)
#Now Run the above cell again to avoid having instance variable school.
```

Output:

```
XYZ
ABC School
```

```
#So if you want to change class data, use:
Student.school="Aror School"
print(s1.school)
print(s2.school)
```

Output:

```
Aror School
Aror School
```

7.Methods: Instance vs class vs Static

```
#Methods are functions inside a class.
#They define behavior of objects.
```

```
#7a. Instance Methods:
    #Work with object data
    #Use self

class Student:

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)

s = Student("Ayesha", 88)
s.display()
```

Output:

Name: Ayesha

Marks: 88

```
# 7b. Class Methods
#Work with class-level data
#Use @classmethod decorator

class College:
    college_name = "ABC University"

    @classmethod
    def show_college(cls, self):
        print(cls.college_name)
```

```
#7c. Static Method

class Student:
    @staticmethod
    def is_passed(marks):
        return marks >= 40

print(Student.is_passed(55))
```

Difference Between Instance, Class, and Static Methods

h17

Feature	Instance Method	Class Method	Static Method
First Parameter	self	cls	None
Decorator Used	None	@classmethod	@staticmethod
Access Instance Data	Yes	No	No
Access Class Data	Yes	Yes	No
Depends on Object	Yes	No	No
Depends on Class	Yes	Yes	No
Main Use	Object behavior	Class-level behavior	Utility / Helper logic

```

class AIModel:

    # Class Variable (shared among all objects)
    total_models = 0

    def __init__(self, name, accuracy):
        # Instance Variables
        self.name = name
        self.accuracy = accuracy

        # Increment class-level counter
        AIModel.total_models += 1

    def improve_accuracy(self, improvement):
        self.accuracy += improvement
        print(f"{self.name} improved to {self.accuracy}%")

    @classmethod
    def get_total_models(cls):
        return cls.total_models

    @staticmethod
    def calculate_f1(precision, recall):
        if precision + recall == 0:
            return 0
        return 2 * (precision * recall) / (precision + recall)

```

```

# Create objects
model1 = AIModel("BrainTumorNet", 85)
model2 = AIModel("LungCancerNet", 90)

# Instance Method (object-specific)
model1.improve_accuracy(3)

# Class Method (class-level)
print("Total Models:", AIModel.get_total_models())

# Static Method (utility)
f1 = AIModel.calculate_f1(0.8, 0.75)
print("F1 Score:", f1)

```

Output:

```
BrainTumorNet improved to 88%  
Total Models: 2  
F1 Score: 0.7741935483870969
```

8. OOP Principles

```
#1. Encapsulation  
#a. Public Members (No Encapsulation)  
class BankAccount:  
    def __init__(self, balance):  
        self.balance = balance  
  
acc = BankAccount(10000)  
  
acc.balance = -5000    # ❌ Dangerous  
print(acc.balance)  
  
#Problems:  
    #Anyone can modify balance  
    #No validation  
    #No security
```

Output:

```
-5000
```

```
#b.Protected Members (_variable): Indicates: "This is for internal use" Not enforced,  
just a warning  
class BankAccount:  
    def __init__(self, balance):  
        self._balance = balance  
  
    def show_balance(self):  
        print("Balance:", self._balance)  
  
acc = BankAccount(5000)  
acc.show_balance()  
  
acc._balance = 100000    # ⚠ Still accessible
```

Output:

```
Balance: 5000
```

```
#c. Private Members (__variable):
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private variable

    def show_balance(self):
        print("Balance:", self.__balance)

acc = BankAccount(8000)

acc.show_balance()    #  Works
# acc.__balance      #  AttributeError
```

Output:

Balance: 8000

Error:

AttributeError: 'BankAccount' object has no attribute '__balance'

```
#d. Getter and Setter Methods
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

    def set_balance(self, amount):
        if amount >= 0:
            self.__balance = amount
        else:
            print("Invalid amount!")

acc = BankAccount(3000)

print(acc.get_balance())

acc.set_balance(5000)
print(acc.get_balance())
acc.set_balance(-2000)    #  Rejected
```

Output:

3000
5000
Invalid amount!
None

```
'''Pythonic Way: Properties (@property)
```

```
Python prefers properties over traditional getters/setters.'''
```

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, amount):
        if amount >= 0:
            self.__balance = amount
        else:
            print("Error")
acc = BankAccount(7000)

print(acc.balance)    # looks like a variable
acc.balance = -9000    # but validated internally
```

Output:

```
7000
Error
```

```
#Practical Example
class ATM:
    def __init__(self):
        self.__pin = 1234

    def verify_pin(self, pin):
        return pin == self.__pin

    def withdraw(self, pin, amount):
        if self.verify_pin(pin):
            print(f"Withdrawn: {amount}")
        else:
            print("Invalid PIN")
```

```
atm = ATM()
atm.withdraw(1234, 2000)
atm.withdraw(1111, 2000)
```

Output:

```
Withdrawn: 2000  
Invalid PIN
```

```
#2. Inheritance:  
#Basic Syntax:  
# class Animal:  
#     # parent class code  
# class Dog(Animal):  
#     # child class code
```

```
#Simple Example:  
class Animal:  
    def speak(self):  
        print("Animal speaks")  
  
class Dog(Animal):  
    def bark(self):  
        print("Dog barks")  
  
d = Dog()  
d.speak()    # Inherited  
d.bark()     # Own method
```

```
#Types of Inheritance in Python  
#Single Inheritance  
#Multiple Inheritance  
#Multilevel Inheritance  
#Hierarchical Inheritance  
#Hybrid Inheritance
```

```
#1.Single Inheritance: One child inherits from one parent.  
class Person:  
    def introduce(self):  
        print("I am a person")  
  
class Student(Person):  
    def study(self):  
        print("I study")
```

#2.Multilevel Inheritance:Grandparent → Parent → Child

```
class Grandparent:
    def show_gp(self):
        print("Grandparent")

class Parent(Grandparent):
    def show_parent(self):
        print("Parent")

class Child(Parent):
    def show_child(self):
        print("Child")
```

Output:

```
Grandparent
Grandparent
Parent
Child
```

#3. Hierarchical Inheritance: One parent → Multiple children

```
class Shape:
    def area(self):
        print("Calculating area")

class Circle(Shape):
    pass

class Rectangle(Shape):
    pass
```

#Multiple Inheritance:

```
class Father:
    def skills(self):
        print("Gardening")

class Mother:
    def talents(self):
        print("Cooking")

class Child(Father, Mother):
    pass
```

```
#Method Overriding
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        print("Dog barks")
```

Output:

```
Animal speaks
Dog barks
```

9. Polymorphism

```
''' Types of Polymorphism:
    Function Polymorphism
    Operator Polymorphism
    Method Overriding (Runtime Polymorphism)
    Duck Typing
    Polymorphism with Inheritance
'''
```

```
#Function Polymorphism
# print(len("Hello"))      # String
# print(len([1,2,3]))      # List
```

```
#Compile time polymorphism is not directly supported but achieved via *args arguments
#Run time polymorphism is supported via method overriding
class Model:
    def predict(self, data):
        pass

class CNN(Model):
    def predict(self, data):
        return "Image classified"

class LLM(Model):
    def predict(self, text):
        return "Generated response"
```

10. Abstraction

```
#Abstraction: Python provides abc module.
#ABC: Abstract Base Class
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

```
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return self.radius*self.radius
```